

HTML5 WebSocket 握手协议的研究与实现

陆 晨 冯向阳 苏厚勤

(东华大学计算机科学与技术学院 上海 201620)

摘 要 WebSocket 规范是作为 HTML5 的一部分提出的,其目标是在客户端浏览器和 Web 服务器之间实现全双工通信。目前主流浏览器已经提供了 WebSocket 客户端 API,但是服务器端并没有标准的 API。对 WebSocket 应用背景进行分析,重点描述 WebSocket 握手协议机制并在 Linux 平台环境下使用 Ruby 编程语言实现一个简单的 WebSocket 服务器。实验证明所描述的研究能够在服务器端成功完成 WebSocket 的握手过程并建立通信连接。

关键词 HTML5 WebSocket 握手协议 Linux Ruby

中图分类号 TP391 **文献标识码** A **DOI:** 10.3969/j.issn.1000-386x.2015.01.033

STUDY AND IMPLEMENTATION OF HTML5 WEBSOCKET HANDSHAKE PROTOCOL

Lu Chen Feng Xiangyang Su Houqin

(School of Computer Science and Technology, Donghua University, Shanghai 201620, China)

Abstract WebSocket specification is proposed as a part of the HTML5, its aim is to realise a full-duplex communication between client browser and Web server. Nowadays, most popular browsers have already provided WebSocket API at client side, but there is no standard API on server site. In this paper we analyse the background of Web applications, elaborately describe the WebSocket handshake protocol mechanism, and implement a simple WebSocket server by using Ruby programming language under Linux platform environment. Experiment proves that the study described in this paper can successfully complete a WebSocket handshake process and establish communication connection on server site.

Keywords HTML5 WebSocket Handshake protocol Linux Ruby

0 引 言

随着计算机网络高速的发展,客户端平台日趋多样化,基于浏览器的应用技术是目前解决应用跨平台的首要方法。而随着 Web 技术在不同领域的应用,人们对用户页面数据的实时性有了较高的要求,因此为解决 Web 即时通信的方案也在不断探索。传统的网站大都是基于轮询的方式,即根据特定的时间间隔,由浏览器向服务器发出 HTTP 请求,比如较早的 HTTP 拉取的方式和较新的 AJAX 技术。为保证用户页面数据的相对即时,每次都需要客户端向 Web 服务器发起 HTTP 请求,然后由 Web 服务器做出响应。这种基于轮询的方式带来了明显的缺点,即需要浏览器不断地向 Web 服务器发起 HTTP 请求,然而这种请求的 Header 是非常长的,这样就增加了不必要的带宽。随着近年来 HTML5 的兴起,特别是互联网主要浏览器厂商如 Google、苹果、Mozilla、Opera 等都大力支持 HTML5 的制定与支持^[1]。作为 HTML5 规范的一部分被提出的 WebSocket 协议^[2]的出现使得浏览器和服务器之间能够建立全双工通信,其实是一种 PUSH 类型^[3]。WebSocket 使得 Web 应用不再需要每次都发起 HTTP 请求来建立与服务端连接来获取数据,而仅需要浏览器与服务器进行一次合法的握手来建立 WebSocket 连接。由于 WebSocket 连接可以实现浏览器和 Web 服务器之间的

全双工通信,因此之后服务器便可以随时地主动向客户端发送更新数据,不需要客户端的频繁请求。

1 WebSocket 应用背景

由于基于浏览器的应用在不同领域的渗透,而 WebSocket 的出现更是在一定程度上扩展了基于浏览器的应用技术的通用性。现有的基于 Web 的监控系统、多媒体信息展示系统、实时聊天系统、可视化电力系统等,都是使用了 Web 应用技术,然而形形色色的领域依然可以基于一个通用的 Web 通信模型来设计。

1.1 基于 WebSocket 的 Web 应用模型

在一个 Web 应用中并不是所有的内容都是需要实时更新的。传统的 Web 即时通信解决方案由于采用 HTTP 协议的通信方式,所以必须基于 HTTP 协议同时实现实时数据模块和非实时数据模块。当使用基于 WebSocket 的解决方案时,实时数据模块和非实时数据模块便可以分别使用 WebSocket 协议和 HTTP 协议来实现,两者各司其职,各自发挥自身的优势互不影响,也降低了系统模块的耦合性^[4],如图 1 描述了基于 WebSocket

收稿日期:2013-04-15。陆晨,硕士生,主研领域:网络通信。冯向阳,副教授。苏厚勤,教授级高工。

的 Web 应用通信模型。

基于 WebSocket 的应用通信模型,浏览器端和服务器端的模块设计可以分为实时模块和非实时模块并分别对应协同工作,两个模块之间互不影响,实时数据可以通过 WebSocket 链路,非实时数据可以依然使用 HTTP 链路,如图 1 所示。

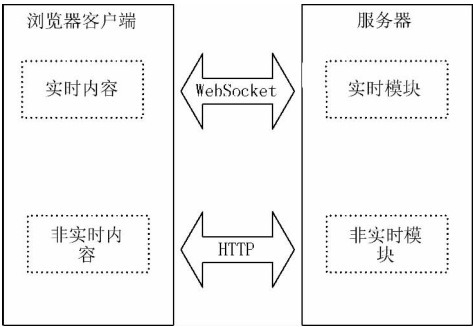


图 1 基于 WebSocket 的 Web 应用模型

1.2 基于 WebSocket 的 Web 应用场景分析

参考现有的即时通信在 Web 应用中的普及解决方案,按照应用需求来分大致可以将这类应用分为实时数据展示和实时数据通信两类,而将 WebSocket 应用于 Web 即时通信时根据浏览器客户端与服务器端的交互频率仍然有相同的分类效果:

(1) 实时数据展示类

此类应用通常只需要服务器不断地将更新的数据发送给客户端并由浏览器展示出来,这个过程中服务器主动发送数据的频率比较高,而客户端的主动交互几乎为零。这类应用如可视化电力系统,多媒体信息展示系统等。

(2) 实时数据通信类

此类应用通常需要浏览器和服务器之间有“对话”动作,既需要客户端向服务器端发送数据,也需要服务器向客户端响应数据。基于 HTTP 的协议同样可以实现这一点,但是 WebSocket 协议的传输格式更加简洁^[4],携带更小的数据量,数据处理过程也更加简便,这类的应用如在线聊天系统,基于 Web 的股票交易系统。

2 WebSocket 握手过程

WebSocket 协议本质上是一个基于 TCP 的协议,WebSocket 连接与 TCP 连接的建立过程类似^[4]。但传统的基于 TCP 连接的协议有所不同的是 WebSocket 协议需要从 HTTP 协议“过渡”而来,而这个“过度”过程也被称为 WebSocket 协议的握手过程。因此想要建立基于 WebSocket 的 Web 应用必须首先了解 WebSocket 协议的握手机制,如图 2 给出了 WebSocket 协议握手过程的示意图。

首先由浏览器客户端向服务器发起 WebSocket 握手请求报文,这个报文是基于 HTTP 协议的,它告诉服务器客户端想要升级当前的 HTTP 协议为 WebSocket 协议。服务器收到客户端的 WebSocket 握手请求报文之后会对报头进行解析,如果服务器理解客户端握手请求报头并且满足升级为 WebSocket 协议的条件,便会向客户端发送握手应答报文,这个应答报文同样是基于 HTTP 协议的。客户端收到服务器的应答报文后会对该报文进行一次验证,验证成功之后便会成功升级为 WebSocket 协议,如果验证失败客户端将会主动断开连接。建立了 WebSocket 连接之后,双方可以进行全双工通信,如图 2 所示。

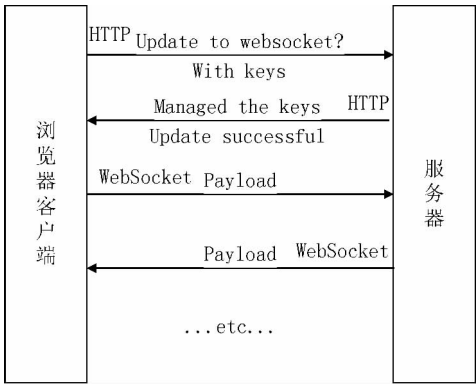


图 2 WebSocket 协议握手过程

3 WebSocket 握手机制

WebSocket 握手过程与 TCP 握手过程类似,但是 WebSocket 协议握手采用了更加简洁的方式。

3.1 WebSocket 握手特点

相对于传统的 TCP 握手,WebSocket 握手协议有以下明显的特点: 首先,WebSocket 整个握手过程只需要两次握手,相对于 TCP 的三次握手,WebSocket 简化并且加入了自己的规则。其次,WebSocket 握手协议是基于 HTTP 协议的,相对于字节流的解析,ASCII 序列解析起来更加简便。最后,WebSocket 握手协议引入了随机序列认证机制,易于实现。

两次握手成功预示着双方接下来将升级当前的 HTTP 协议为 WebSocket 协议。WebSocket 握手请求报文的报头部分除了必须包含必要的 HTTP 字段^[5],还须遵循通用消息格式 [RFC 822],同时又要包含和 WebSocket 紧密联系的字段^[5],如 WebSocket 协议的版本信息、客户端随即生成的 Key、必要的 GET 请求方法等。

3.2 客户端握手请求报文

基于 HTTP 协议的握手请求报文报头部分除了需要包含必要的 HTTP 字段以外,还需要包含 WebSocket 协议定义的字段。WebSocket 客户端握手报头字段名、必要和选项字段和字段对应的用途如表 1 所示^[5]。

表 1 WebSocket 客户端请求报头有关字段

字段名	选项否	用途
Request-line	必要	HTTP 请求行,其中请求方法必须是 GET,HTTP 版本至少是 1.1 ^[5]
HOST	必要	请求目标主机域名/host/,加上可选的“:”后跟端口号/port/
Upgrade	必要	标准规定必须包含 Upgrade 头域,且值必须是大小写不敏感的 ASCII 值“websocket”
Connection	必要	标准规定必须包含 Connection 头域,且值必须是大小写不敏感的 ASCII 值“Upgrade”
Sec-WebSocket-Key	必要	其值必须是由随机数组成的随机选择的 16 字节的被 base64 编码后的值

续表 1

字段名	选项否	用途
Origin	基于浏览器请求时必要	当请求来自浏览器时,其值是建立连接代码运行环境的域名 ASCII 序列
Sec-WebSocket-Version	必要	标准规定必须包含 Sec-WebSocket-Version 头域,其值必须是 13
Sec-WebSocket-Protocol	选项	如果头域包含字段,此值指示客户端希望使用的一个或多个逗号分隔的子协议,按优先顺序
Sec-WebSocket-Extensions	选项	如果有头域包含该字段,其值指示了客户端希望使用的协议级别的扩展

此外,请求报文报头部分还可能包含任意的其他 HTTP 字段,如 Cookie 字段等。

3.3 服务器端握手响应报文

基于 HTTP 协议的 WebSocket 服务器握手响应报文报头字段除需包含必要的 HTTP 报头字段之外,还需要包含 WebSocket 协议定义的报头字段。WebSocket 服务器端握手报文字段名、必要和选项字段和每个字段对应的用途如表 2 所示^[8]。

表 2 WebSocket 服务器端响应报头有关字段

字段名	选项否	用途
Status-line	必要	HTTP 请求响应状态行,状态码必须是 101
Upgrade	必要	标准规定必须包含 Upgrade 头域,且值必须是大小写不敏感的 ASCII 值“websocket”
Connection	必要	标准规定必须和 Upgrade 头域完成 HTTP 升级
Sec-WebSocket-Accept	必要	标准规定必须包含 Sec-WebSocket-Accept 头域,该头域表明服务器是否愿意接受客户端连接,且其值必须是按照既定算法得到的值,否则不能解释为服务器接受连接
Sec-WebSocket-Extensions	选项	如果包含该头域,其值指示服务器是否支持客户端希望使用的协议级别扩展
Sec-WebSocket-Protocol	选项	表示服务器选择的子协议

握手能否成功关键过程为:

首先由客户端发送一个随机的基于 base64 编码的 16 字节序列,即客户端请求字段中的 Sec-WebSocket-Key 字段的值。服务器端解析 HTTP 请求报头获取 Sec-WebSocket-Key 字段值后根据既定算法生成响应序列,即为服务器端响应报头的 Sec-WebSocket-Accept 字段的字段值,然后由客户端验证服务器响应的序列值,验证成功则同意握手成功并建立连接。

4 WebSocket JavaScript 接口

针对浏览器客户端构建 WebSocket 应用非常简单,因为

JavaScript 为此提供了一套 WebSocket 应用程序接口,而越来越多的浏览器已经对这些接口进行了支持^[9]。下面对 JavaScript 提供的 WebSocket 主要接口进行简单介绍^[8]。

(1) WebSocket (url, protocols)

用于构造 WebSocket 对象,url 参数指向连接的目标 url, protocols 参数用于指定双方建立的连接所使用 WebSocket 的子协议名称。

(2) void send (data)

用于向服务器发送数据,发送数据 data 的类型有 DOMString, Blob, ArrayBuffer 和 ArrayBufferView 四种。

WebSocket JavaScript API 采用事件触发的方式实现方法调用,其中条件与对应触发事件关系如图 3 所示。

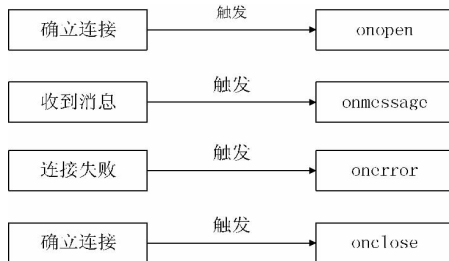


图 3 事件源与触发事件关系图

若客户端验证服务器端的握手响应报文之后同意握手成功,就会触发 onopen 事件。

5 WebSocket Server 的实现

5.1 Sec-WebSocket-Accept 生成算法描述

WebSocket 协议 [RFC6455] 规定 Sec-WebSocket-Accept 字段生成的算法描述如下^[8]:

a) 将客户端的 Sec-WebSocket-Key 字段与 WebSocket 既定全球唯一标识符 (GUID, [RFC4122]) "258EAF5E914-47DA-95CA-C5AB0DC85B11" 以字符串的形式连接。

b) 将连接的字符串使用 SHA-1 (160 bits) [FIPS. 180-3] 算法散列。

c) 将上一步散列结果进行 base64 编码。

经过上述三步之后将成功生成服务器端握手响应报文的 Sec-WebSocket-Accept 字段值。

5.2 WebSocket Server 通信模型设计

本文设计 WebSocket 客户端与服务器的通信模型采用 Ruby Socket 和 Ruby 多线程技术,客户端和服务端之间的通信模型如图 4 所示。

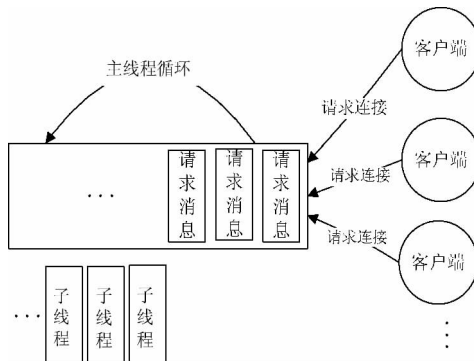


图 4 WebSocket 客户端-服务器通信模型

该模型的好处在于可以实现多点同时与服务器发起 WebSocket 握手请求。

5.3 基于 Ruby 的 WebSocket Server 实现

Ruby 是一种简单快捷的面向对象的脚本语言^[8], 它有丰富的类库, 并完美支持 Md5、SHA1 和 Base64 算法, 以及支持 Socket 网络接口编程。本文在 Linux 平台下使用 Ruby 语言完成 WebSocket 握手协议的服务端实现。

定义主要接口为: `def handshake( client )`。

该接口主要功能是完成 WebSocket 服务器端对于客户端握手请求报文的理解并给与响应。其中 `Sec-WebSocket-Accept` 字段的生成, 使用 Ruby 类库提供的方法实现起来非常简便, 代码语句为:

```
Sec-WebSocket-Accept =  
Base64.strict_encode64( Digest::SHA1.digest( key ) )
```

如上所示, 仅仅使用一行语句即可。

WebSocket 服务器端实现的过程:

首先接收客户端的 TCP 连接请求, 然后从输入流中读取 HTTP 协议报文并解析, 获取客户端填充的 `Sec-WebSocket-Key` 字段值, 根据该字段值按照既定算法生成正确的序列并填充服务器握手响应报文头域中的 `Sec-WebSocket-Accept` 字段。

`handshake( client )` 接口的实现代码如下:

```
def handshake( client )  
  guid = "258EAF5 - E914 - 47DA - 95CA - C5AB0DC85B11"  
  line = client.gets  
  while line = client.gets  
    k,v = line.split( ":" ), v = v.strip  
    if k == "Sec-WebSocket-Key"  
      key = Base64.strict_encode64( Digest::SHA1.digest( v +  
guid ) )  
    end  
  end  
  client.print "HTTP/1.1 101 Web Socket Protocol Handshake \r\n"  
  
  client.print "Upgrade: WebSocket\r\n"  
  client.print "Connection: Upgrade\r\n"  
  client.print "Sec-WebSocket-Accept: " + key + "\r\n\r\n"  
end
```

服务器端按照上文介绍的通信模型完成主方法实现。

WebSocket 服务器端主方法实现如下:

```
server = TCPServer.open( 80 )  
loop{  
  Thread.start( server.accept ) do |client|  
    handshake( client )  
  end  
}
```

以上代码可以看出 Ruby 实现 WebSocket 服务器握手协议拥有较少的代码量, 实现起来非常简便, 利用 Ruby Socket 和 Ruby 多线程技术实现的通信模型可以让服务器端同时和多个客户端握手。

此外, 试验首先在客户端通过上文介绍的 WebSocket JavaScript API 实现一个简单的 WebSocket 客户端, 其核心代码如下:

```
var url = "ws://localhost"  
websocket = new WebSocket( url );  
websocket.onopen = function( evt ) { onOpen( evt ) };  
function onOpen( evt ) {
```

```
  alert( "握手成功!" );
```

```
}
```

本文因为仅仅需要验证握手是否成功, 因此只需要 WebSocket JavaScript API 部分接口即可。

6 实践

为验证本文的研究结论, 本实践通过 Chrome 浏览器运行 WebSocket JavaScript 客户端, 即将 WebSocket JavaScript 客户端脚本嵌入 HTML 页面, 然后请求这个页面。WebSocket JavaScript 客户端访问本机地址为 localhost 的 WebSocket 服务器, 缺省 Web 服务器通信端口为 80。

使用截图工具在利用 Chrome 浏览器的审查元素小工具查看报头信息时截取了客户端浏览器的请求报头和 WebSocket 服务器端的响应报头内容, 如图 5 和图 6 所示。从图中可以看到客户端请求报头的报头域和 WebSocket 服务器端响应的报头域。



图 5 WebSocket 握手请求 Headers

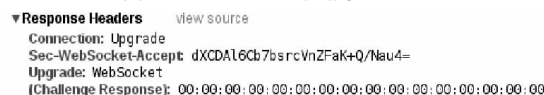


图 6 WebSocket 握手响应 Headers

客户端请求 WebSocket 服务器时使用的 url 参数为 “ws://localhost/”, 其中前缀 “ws:” 为 WebSocket 标准定义的模式之一, 另一种模式是 “wss:”, 服务器成功返回状态码 101, 如图 7 所示。同时 onopen 事件被成功触发, 弹出 “握手成功!” 字样。说明此时客户端和服务器之间握手成功, 通信协议已经成功升级为 WebSocket 协议, 本实践结果表明根据本文研究并实现的 WebSocket 服务器可以顺利完成 WebSocket 握手过程。

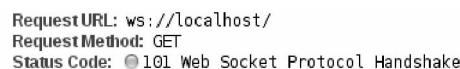


图 7 HTTP 请求及响应信息

7 结 语

本文借助 Ruby 语言在 Linux 平台下实现了一个简单的 WebSocket 服务器, 并通过实验验证该服务器可以配合浏览器提供的 WebSocket API 成功完成 WebSocket 的握手过程。Ruby 语言风格简洁, 等同功能有更少的代码量。Ruby 在 Linux 平台下易于完成安装和使用, 并且不依赖于 Web 服务器, 因此适合于基于 Linux 的嵌入式设备, 是一个值得研究和实践的重点之一。

参 考 文 献

- [1] 孙松柏, Ali Abbasi, 诸葛建伟, 等. HTML5 安全研究 [J]. 计算机应用与软件, 2013, 30(3): 1-6, 16.

(下转第 178 页)

表 4 不同算法识别率对比(正脸、遮挡)

覆盖位置/方法	Gabor	Resize	LBP	本文算法
眼	22.06%	69.12%	57.35%	95.59%
口	42.65%	83.82%	50.00%	94.12%
鼻	27.94%	86.76%	77.94%	100.00%

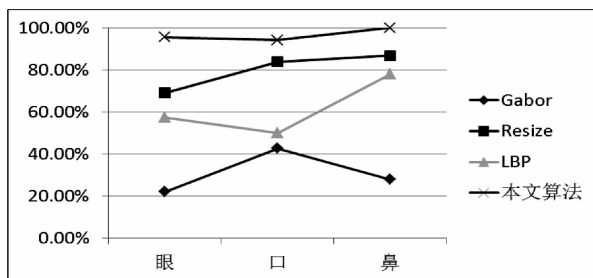


图 10 不同算法识别率对比(正脸、遮挡)

表 5 不同算法识别率对比(侧脸、遮挡)

覆盖位置/方法	Gabor	Resize	LBP	本文算法
眼	80.88%	86.76%	70.59%	94.12%
口	88.24%	86.76%	73.53%	97.06%
鼻	83.82%	86.76%	72.06%	95.59%

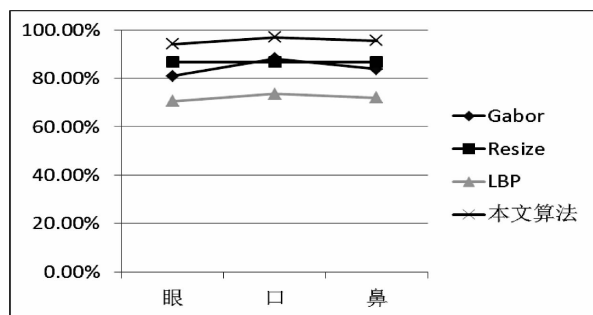


图 11 不同算法识别率对比(侧脸、遮挡)

表 6 不同算法识别率对比(斜侧脸、遮挡)

覆盖位置/方法	Gabor	Resize	LBP	本文算法
眼	28.79%	81.82%	72.73%	100.00%
口	62.12%	90.91%	78.79%	98.48%
鼻	65.15%	90.91%	92.42%	100.00%

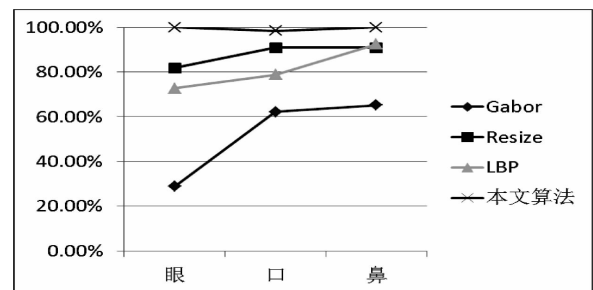


图 12 不同算法识别率对比(斜侧脸、遮挡)

通过上述的实验结果可以看到,本文的方法在人脸图像存在遮挡的情况下,识别率接近 100%,而其他 3 中情况则相较无遮挡的情况,出现了明显的下滑。其次,可以看到在遮挡的情况下,LBP 比 Gabor 具有更高的识别率,说明 LBP 能比 Gabor 更好地保留人脸的结构信息。最后,可以看到 2 种使用稀疏编码的算法(本文算法和 Resize),都具有较高的识别率。通过本文实验,验证了本文的算法不仅具有较高的识别率,同时也具有更好的鲁棒性。

5 结 语

本文对于人脸识别应用中普遍存在的样本过少的问题,提出了一种基于分块均匀 LBP 算子和稀疏编码求解的人脸识别算法,解决了维数约减算法失效的问题。同时均匀 LBP 算子具有计算简便、较好保留人脸局部特征的优势,相比其他的人脸特征,例如 Gabor 特征,有更高的识别率。本文还通过枚举不同的分块方法,得出 4×4 的分块方法最适合人脸结构。

参 考 文 献

- [1] Bhuiyan A, Liu C. On Face Recognition using Gabor Filters [J]. WA-SET, 2007, 2(1): 51-56.
- [2] Kepenekci A T S, et al. ABSTRACT FACE RECOGNITION USING GABOR WAVELET, (September) [R].
- [3] Matos, et al. Face Recognition Using DCT Coefficients Selection [R]. 2008, 1753-1757.
- [4] Hafed Z M, Levine M D. Face Recognition Using the Discrete Cosine Transform, [J]. International Journal of Computer Vision, 2001, 43(3): 167-188.
- [5] Ojala T, Pietikainen M. Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns [J]. Pattern Analysis and Machine Intelligence, 2002, 24(7): 971-987.
- [6] Ahonen T, Hadid A, Pietikainen M. Face description with local binary patterns: application to face recognition [J]. IEEE transactions on pattern analysis and machine intelligence, 2006, 28: 2037-2041.
- [7] Wright J, Yang A Y. Face Recognition via Sparse Representation Research About Face [J]. Pattern Analysis and Machine Intelligence, 2008, 31(2): 210-217.
- [8] Shan, Caifeng. Learning local binary patterns for gender classification on real-world face images [J]. Pattern Recognition Letters, 2012, 33(4): 431-437.
- [9] Maturana D, Mery D, Soto A. Face recognition with decision tree-based local binary patterns [M]. // Computer Vision - ACCV 2010. Springer Berlin Heidelberg, 2011: 618-629.
- [10] Wagner A, Wright J, Ganesh A, et al. Toward a practical face recognition system: Robust alignment and illumination by sparse representation [J]. Pattern Analysis and Machine Intelligence, IEEE Transactions on, 2012, 34(2): 372-386.

(上接第 131 页)

- [2] WebSocket.org. About WebSocket [EB/OL]. <http://www.websocket.org/aboutwebsocket.html>, 2013.
- [3] 龙奇. 下一代 Web 通信技术 HTML5 WebSocket 的研究 [J]. 科技信息, 2011, 11(36): 273.
- [4] 温照松, 易仁伟, 姚寒冰. 基于 WebSocket 的实时 Web 应用解决方案 [J]. 电脑知识技术, 2012, 8(16): 3826-3828.
- [5] Hypertext Transfer Protocol—HTTP/1.1 [S/OI]. 2013. <http://www.ietf.org/rfc/rfc2616.txt>.
- [6] The WebSocket Protocol RFC 6455 [S/OI]. 2013. http://datatracker.ietf.org/doc/rfc6455/?include_text=1.
- [7] 李代立, 陈榕. WebSocket 在 Web 实时通信领域的研究 [J]. 电脑知识与技术, 2010, 6(28): 7923-7925, 7935.
- [8] W3C. The WebSocket API [EB/OL]. 2013. <http://dev.w3.org/html5/websockets/>.
- [9] 隗刚, 张欣, 裴宗燕. Ruby 在网络上的应用 [J]. 程序员, 2003(1): 84-86.